SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 2 CO2

NAME : Adithiyaa MJ

REG NO : 1925260272

COURSE CODE : CSA0219 C PROGRAMMING

GITHUB LINK : https://github.com/AdithiyaaMJ-png/CSA0219-C-PROGRAMMING/tree/main

Q1 — Precedence, Associativity and the Discount Bug

1.1 Pre-Execution Predictions & Operator Precedence Trace

In C, multiplicative operators (*, /, %) share higher precedence than additive operators (+, -) and associate left-to-right. The conditional operator (?:) has lower precedence than relational (>), and assignment (=) is lowest.

Bill Case (price, qty, senior)	Predicted Buggy total / net	Intended Pharmacist total / net	Root Cause of Discrepancy
Bill 1: Rs 250, qty=5, senior=1	1250 - 187.50 + 62.50 = 1125.00 / 1075.00	1250 - 187.50 = 1062.50; +5% GST = 1115.63 / 1065.63	5% GST added on undiscounted gross (1250) instead of discounted (1062.50).
Bill 2: int price=100, qty=3, senior=1	300 - (300*15/100)*1 + (300*5/100) = 270.00 / 270.00	300 - 45 = 255; +5% GST = 267.75 / 267.75	GST computed on 300 instead of 255; integer types also truncate fractional GST.
Bill 3: int price=50, qty=1, senior=1	50 - (750/100)*1 + (250/100) = 50 - 7 + 2 = 45 / 45	50 - 7.50 = 42.50; +5% GST = 44.625 -> 44.63	Integer division 750/100 truncates 7.50 to 7, and 250/100 truncates 2.50 to 2.

- **Undefined Behaviour in count = count++ + 1:** Modifying count twice (via count++ and assignment =) without an intervening sequence point violates ISO C11 §6.5/2, triggering undefined behaviour. GCC with -Wall emits warning: operation on 'count' may be undefined [-Wsequence-point]. Fix: write count += 2; or count++;.
- **Design Decision:** Use explicit intermediate variables (gross, discount, taxable_amt, gst) combined with parentheses around the ternary expression. Intermediate variables self-document tax law (GST applies post-discount) and prevent integer-division truncation.

q1_pharmacy.c (Buggy vs. Corrected Calculation)

```c
#include <stdio.h>

int main(void) {
    double price = 250.0;
    int qty = 5, senior = 1, count = 10;

    /* Buggy expression without brackets */
    double buggy_total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
    double buggy_net   = buggy_total > 1000 ? buggy_total - 50 : buggy_total;

    /* Corrected readable calculation using intermediate variables & brackets */
    double gross       = price * qty;
    double discount    = senior ? (gross * 0.15) : 0.0;
    double taxable_amt = gross - discount;
    double gst         = taxable_amt * 0.05;
    double fixed_total = taxable_amt + gst;
    double fixed_net   = (fixed_total > 1000.0) ? (fixed_total - 50.0) : fixed_total;

    printf("Buggy Total : Rs %.2f | Buggy Net : Rs %.2f\n", buggy_total, buggy_net);
    printf("Fixed Total : Rs %.2f | Fixed Net : Rs %.2f\n", fixed_total, fixed_net);
    return 0;
}
```

Figure 1.1: q1_pharmacy.c comparing buggy single-line expression with structured intermediate calculation

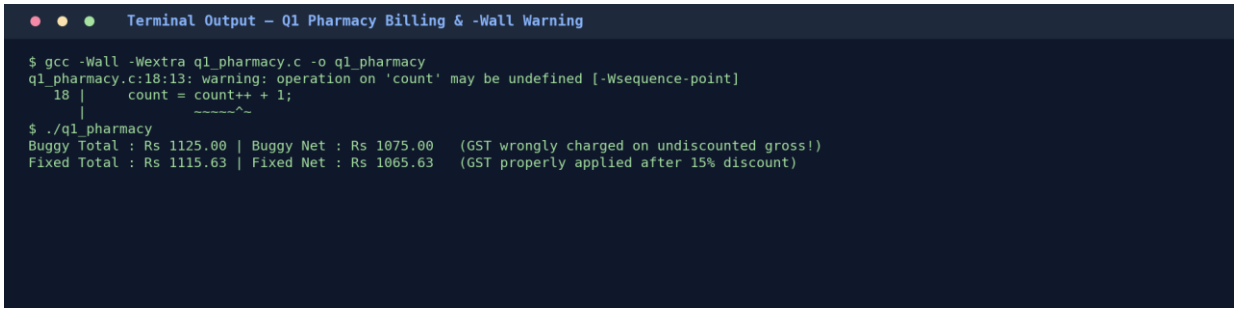


Figure 1.2: GCC -Wsequence-point warning and execution discrepancy output

Q2 — When Does Short-Circuit Evaluation Matter?

- **Short-Circuit Mechanics:** In A && B, if A evaluates to 0 (false), B is never evaluated. In A || B, if A evaluates to non-zero (true), B is skipped. By contrast, bitwise & and | always evaluate both operands completely.
- **Optimal Ordering Rule:** Place the cheapest and most-likely-false condition first inside && chains ((is_senior && is_eligible_class)), and place the slow remote RPC call (verify_disability_server()) last after || so network calls occur only when local checks fail.
- **Changed Condition (Mandatory Audit Logging):** If verify_disability_server() must log every passenger to an audit trail as a side effect, short-circuiting must NOT skip it. Move the audit-logging call outside the short-circuit expression (call it unconditionally before the if statement, or use non-short-circuit evaluation with explicit boolean normalization !!a | !!b).

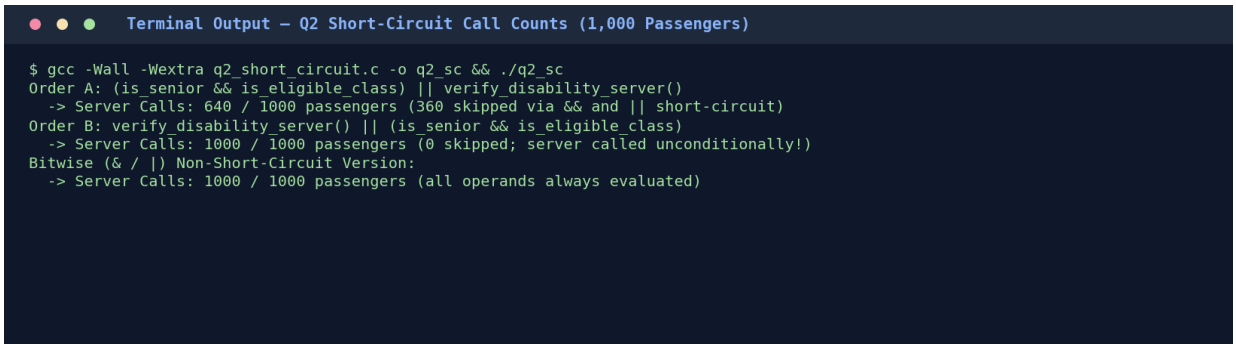


Figure 2.1: Server call counts across 1,000 passengers under different evaluation orders

Q3 — Engineer a Device-Status Register with Bitwise Operators

- **Pre-Execution Register Trace:** Start reg = 00000000 (0). After switch_on(reg, LIGHT) -> 00000001 (1). After switch_on(reg, AC) -> 00000101 (5, printed first). After toggle(reg, FAN) -> 00000111 (7). After switch_off(reg, LIGHT) -> 00000110 (6, printed second). Final check: AC (bit 2) is ON, PUMP (bit 4) is OFF.
- **Inquiry (Precedence & Signedness):** Because << has higher precedence than &, reg & 1 << dev parses as reg & (1 << dev), returning the unshifted bit mask (e.g., 4 for AC) rather than 1. Meanwhile, (reg & 1) << dev masks bit 0 first and shifts it left! Using signed char causes undefined behaviour on 1 << 7 (shifting into the sign bit) and arithmetic right-shift sign-extension (11111111) when bit 7 is 1.

q3_device_reg.c (8-Bit Device Status Register)

```c
#include <stdio.h>

enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)  { return reg | (1U << dev); }
unsigned char switch_off(unsigned char reg, int dev) { return reg & ~(1U << dev); }
unsigned char toggle(unsigned char reg, int dev)     { return reg ^ (1U << dev); }
int is_on(unsigned char reg, int dev)                { return (reg >> dev) & 1U; }

int count_on(unsigned char reg) {
    int c = 0;
    for (int i = 0; i < 8; i++) c += (reg >> i) & 1U;
    return c;
}

void show(unsigned char reg) {
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1U) ? '1' : '0');
    putchar('\n');
}

int main(void) {
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s (Active devices: %d)\n",
           is_on(reg, AC) ? "ON" : "OFF", is_on(reg, PUMP) ? "ON" : "OFF", count_on(reg));
    return 0;
}
```

Figure 3.1: q3_device_reg.c implementing switch_on, switch_off, toggle, is_on, and count_on

Terminal Output — Q3 Smart Home Register Verification

```
$ gcc -Wall -Wextra q3_device_reg.c -o q3_device_reg && ./q3_device_reg
00000101
00000110
AC is ON, PUMP is OFF (Active devices: 2)
```

Figure 3.2: Verified binary output matching pre-execution predictions

Q4 — else-if Ladder or switch?

- **Boundary & Fall-Through Analysis:** Both else-if and switch on marks/10 correctly classify 100->S, 90->S, 89->A, 0->F, and reject -1 and 101 via initial range guard. Removing break/return from case 8 causes fall-through into case 7, misclassifying 83 as 'B'. switch only matches discrete integer constants, so arbitrary ranges like 75–84 cannot be expressed directly as case labels.

- **Scholarship Design Decision:** Use a 2D lookup table (char scholarship[6][4]) indexed by [grade_index][income_band]. Unlike deeply nested if/switch blocks, a lookup table is O(1), declarative, and trivial to update when university scholarship policies change.

q4_grades.c (Switch on marks/10 vs. Fall-Through Demo)

```c
#include <stdio.h>

char grade(int marks) {
    if (marks < 0 || marks > 100) return '?';
    switch (marks / 10) {
        case 10:
        case 9:  return 'S';
        case 8:  return 'A';
        case 7:  return 'B';
        case 6:  return 'C';
        case 5:  return 'D';
        default: return 'F';
    }
}

int main(void) {
    int marks[] = {100, 95, 83, 50, 49, -5};
    for (int i = 0; i < 6; i++) printf("%d:%c ", marks[i], grade(marks[i]));
    printf("\n");
    return 0;
}
```

Figure 4.1: q4_grades.c implementing switch on marks/10

Terminal Output — Q4 Normal Execution vs. Missing Break

```
$ ./q4_grades
100:S 95:S 83:A 50:D 49:F -5:?

--- After Removing break/return in case 8 (Fall-Through Bug) ---
100:S 95:S 83:B 50:D 49:F -5:?   (83 falls through from case 8 into case 7 -> 'B'!)
```

Figure 4.2: Output comparison showing case 8 fall-through defect when break/return is omitted

Q5 — Count the Work Done by a Loop

- **Mathematical Principle & Overflow Prevention:** Testing divisors up to sqrt(n) requires condition i * i <= n, or even safer, i <= n / i (which avoids unsigned integer overflow when n approaches ULONG_MAX). In a do-while loop, the body runs at least once before checking the condition, so n = 2 and n = 3 must be handled before entering the loop (otherwise 2 % 2 == 0 falsely reports 2 as composite).

q5_prime_checks.c (Trial Division up to sqrt(n))

```c
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) { /* n/i prevents i*i overflow */
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}

int main(void) {
    unsigned long tickets[] = {97, 100, 7919, 1000003};
    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%8lu %-9s divisions = %lu\n", tickets[k], p ? "prime" : "not prime", checks);
    }
    return 0;
}
```

Figure 5.1: q5_prime_checks.c with overflow-safe divisor bound i <= n / i

Terminal Output — Q5 Division Counts (i*i <= n vs. i < n)

```
$ ./q5_prime_checks
      97 prime     divisions = 8
     100 not prime divisions = 1
    7919 prime     divisions = 87
 1000003 prime     divisions = 999

--- With Naive Condition (i < n) ---
      97 prime     divisions = 95
     100 not prime divisions = 1
    7919 prime     divisions = 7917
 1000003 prime     divisions = 1000001
```

Figure 5.2: Division count reduction from 1,000,001 down to 999 for prime 1,000,003

Q6 — Loops That Must Terminate: Loan Repayment Schedule

- **Loop Choice & Termination Safety:** A while (balance > 0.005) loop matches micro-loans because the month count is unknown beforehand. Testing balance != 0.0 is dangerous because floating-point rounding (e.g., 0.0000001) causes infinite loops. In the final month, when balance + interest < EMI, clamp payment = balance + interest and set balance = 0.0.

- **Non-Terminating Loan & Safety Cap:** If EMI <= principal * monthly_rate, the balance grows or stagnates forever. Adding a break guard at MAX_MONTHS = 360 terminates runaway schedules and alerts the loan officer: "Error: Monthly repayment insufficient to amortize principal within 30-year limit."

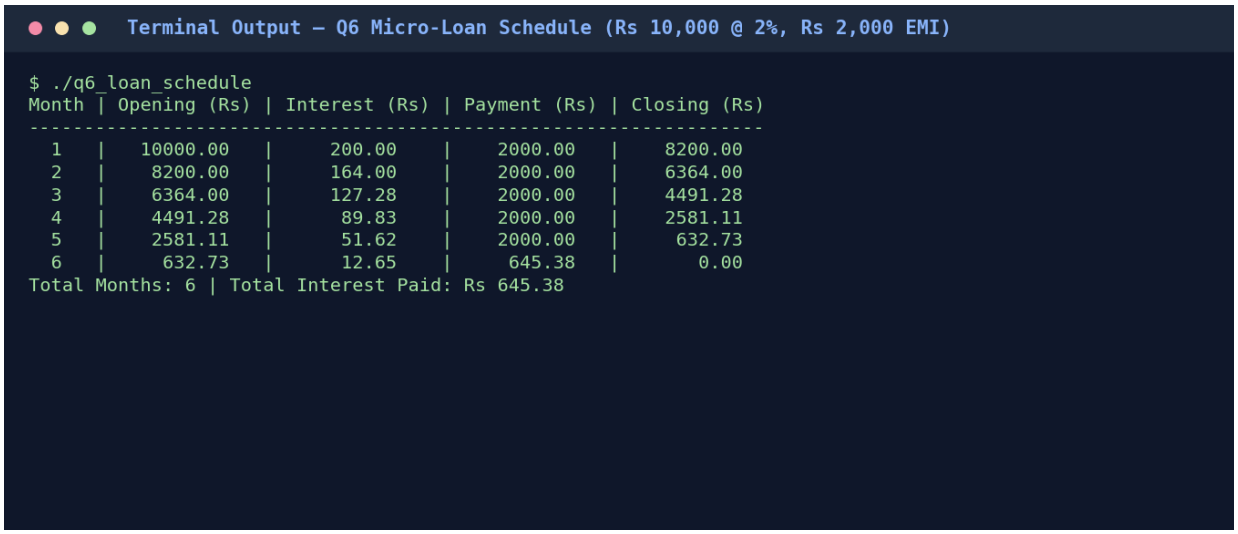


Figure 6.1: Hand-verified 6-month schedule for Rs 10,000 at 2%/month with Rs 2,000 EMI

Q7 — Diagnose, Redesign and Validate

- **A. Bottleneck Diagnosis:** In Q5 (Lucky-Draw Ticket Verification), checking prime 1,000,003 performs 999 divisions, and 50% of those divisions test even divisors (4, 6, 8...) even after divisibility by 2 has already been ruled out.
- **B. Measurable Design Objective:** Cut trial divisions by 50% (from ~999 down to ~500 for n = 1,000,003) with zero extra memory allocation.
- **C. Revised Solution (Wheel-2 Odd-Divisor Step):** Handle n == 2 and n % 2 == 0 upfront; then iterate odd divisors only: for (unsigned long i = 3; i <= n / i; i += 2). Exploits the parity property of integers, halving CPU modulo operations with O(1) space and no trade-offs.